

locals() and globals()

- locals() returns a dictionary containing local variables of the function.
- globals() returns a dictionary of global variables and function details.

Example:

```
a = 10 # global variable
def my_fun():
    b = 20 # local variable

    print(locals())
    print(globals())

my_fun()
```

Output:

```
locals() # {'b': 20}

globals() # {'__name__': '__main__', 'a': 10, 'my_fun': <function my_fun at 0x...>}
```

zip()

It is a built-in Python function that combines multiple lists (or iterables) element by element into tuples based on their positions.

Syntax : zip(iterable1, iterable2, ...)

Example : with 2 list

```
names = ["Alice", "Bob", "Charlie"]
marks = [90, 85, 95]
res = zip(names, marks)
print(res)
```

Output: <zip object at 0x7f9b6c1d0>

Note:

It does not directly return a list or tuple.

The zip() function returns a zip object.

We can convert it into a list, tuple, set or dict using list(), tuple(), set() or dict().

Converting to list:

```
res = list(zip(names, marks))  
print(res)
```

Output: [('Alice', 90), ('Bob', 85), ('Charlie', 95)]

Converting to tuple:

```
res = tuple(zip(names, marks))  
print(res)
```

Output: (('Alice', 90), ('Bob', 85), ('Charlie', 95))

Converting to dict:

```
keys = ["name", "age", "grade"]  
values = ["Alice", 15, "A"]  
std_details = dict(zip(keys, values))  
print(std_details)
```

Output : {'name': 'Alice', 'age': 15, 'grade': 'A'}

zip() + dict() → create dictionaries quickly.

Example: with 3 list

```
names = ["Alice", "Bob", "Charlie"]  
marks = [90, 85, 95]  
grades = ["A", "B", "A"]  
result = zip(names, marks, grades)  
print(list(result))
```

Output: [('Alice', 90, 'A'), ('Bob', 85, 'B'), ('Charlie', 95, 'A')]

Example : Lists of Unequal Length

```
names = ["Alice", "Bob", "Charlie"]
marks = [90, 85]
result = zip(names, marks)
print(list(result))
```

Output: [('Alice', 90), ('Bob', 85)]

zip() stops when the shortest list ends.

Example : with strings

```
str1 = "ABC"
str2 = "123"
res = list(zip(str1, str2))
print(res)
```

Output : [('A', '1'), ('B', '2'), ('C', '3')]

Example: List + Tuple

```
names = ["Alice", "Bob", "Charlie"]
marks = (90, 85, 95)
res = list(zip(names, marks))
print(res)
```

Output: [('Alice', 90), ('Bob', 85), ('Charlie', 95)]

zip() does not care about the type, only that they are iterable.

Note:

- It is not possible to convert a zip object to string directly.
 - First we need to convert to list or tuple and then we need to convert to string.
-

Step 1 — Convert zip object to list of tuples

```
names = ["Alice", "Bob", "Charlie"]
marks = (90, 85, 95)
zipped = list(zip(names, marks))
```

Step 2 — Convert list of tuples to string

```
res = str(zipped)
print(res)
```

Output: "[('Alice', 90), ('Bob', 85), ('Charlie', 95)]"

Zip Keys of One dict with Another: Combines keys from dict1 with dict2.

```
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"x": 10, "y": 20, "z": 30}
res = zip(dict1, dict2)
print(dict(res))
```

Output : {'a': 'x', 'b': 'y', 'c': 'z'}

Zip Keys of One Dict with Values of Another

```
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"x": 10, "y": 20, "z": 30}
zipped = zip(dict1.keys(), dict2.values())
res_dict = dict(zipped)
print(res_dict)
```

Output: {'a': 10, 'b': 20, 'c': 30}

Zip Keys of Two Dicts

```
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"x": 10, "y": 20, "z": 30}
print(list(zip(dict1.keys(), dict2.keys())))
```

Output: [('a', 'x'), ('b', 'y'), ('c', 'z')]

Zip Values of Two Dicts

```
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"x": 10, "y": 20, "z": 30}
zipped_values = zip(dict1.values(), dict2.values())
print(list(zipped_values))
```

Output: [(1, 10), (2, 20), (3, 30)]

Create a New Dict from Two Dicts

```
students = {"Alice": "Math", "Bob": "Science"}
marks = {"Alice": 95, "Bob": 85}
student_marks = dict(zip(students.keys(), marks.values()))
print(student_marks)
```

Output: {'Alice': 95, 'Bob': 85}

Using zip for 3 lists (not possible directly)

```
students = ["Alice", "Bob", "Charlie"]
marks = [95, 85, 90]
grades = ["A", "B", "A"]
res = dict(zip(students, marks, grades))
```

#The above example gives Error

Using zip for 3 lists (value as tuple)

```
students = ["Alice", "Bob", "Charlie"]
marks = [95, 85, 90]
grades = ["A", "B", "A"]
res = dict(zip(students, zip(marks, grades)))
print(res)
```

Output: {'Alice': (95, 'A'), 'Bob': (85, 'B'), 'Charlie': (90, 'A')}

Packing

- Packing means putting multiple values into a single variable.
 - Usually done with tuples, but can work with lists too.
 - Think of it as putting items in a backpack.
-

Basic Tuple Packing:

Multiple values packed into one variable

```
student = "Alice", 95, "A"  
print(student)  
print(type(student))
```

Output:

```
('Alice', 95, 'A')  
<class 'tuple'>
```

List Packing

```
student = ["Bob", 85, "B"] # Packed into a list  
print(student)  
print(type(student))
```

Output:

```
['Bob', 85, 'B']  
<class 'list'>
```

Packing Multiple Variables

```
a, b, c = 10, 20, 30  
packed = a, b, c  
print(packed)
```

Output: (10, 20, 30)

Packing in Function Return

```
def student_info():  
    name = "Charlie"  
    marks = 90  
    grade = "A"  
    return name, marks, grade # Packing 3 values in one return
```

```
info = student_info()  
print(info)
```

Output: ('Charlie', 90, 'A')

Packing in Function with *arg

```
def greet(*names):  
    print(names)  
greet("Bob", "Charlie", "David")
```

Output: ('Bob', 'Charlie', 'David')

Unpacking

It means taking values from a collection (like list, tuple, dictionary) and assigning them to variables.

Basic Example (Tuple Unpacking)

```
data = (10, 20, 30)  
a, b, c = data  
print("a: ", a)  
print("b: ", b)  
print("c: ", c)
```

Output

```
a: 10  
b: 20  
c: 30
```

List Unpacking

```
numbers = [1, 2, 3]
x, y, z = numbers
print(x, y, z)
```

Output : 1 2 3

Using * for Unpacking

```
nums = [10, 20, 30, 40]
a, *b = nums
print("a: ", a)
print("b: ", b)
```

Output:

```
a: 10
b: [20, 30, 40]
```

* collects the remaining values.

Function Unpacking

```
def add(a, b, c):
    print(a + b + c)
```

```
values = (1, 2, 3)
add(*values)
```

Output : 6
